using:

$$M = E - N + 2P$$

Where:

E = Number of edges
N = Number of nodes
P = Number of connected components

Simplified:

M = Number of decision points + 1
Determining Independent Paths
Cyclomatic Complexity gives the number of linearly independent paths
Each path represents a unique execution route

Example:
If a CFG has 3 decision points:

$$M = 3 + 1 = 4$$

So, 4 independent paths must be

tested.

Importance in Test Case Design
Minimum Test Cases
Ensures at least M test cases for full path coverage
Improved Coverage
Covers all branches and conditions
Detection of Logical Errors
Helps identify hidden bugs in complex logic
Code Quality Indicator
High complexity (>10–20) indicates risky and hard-to-maintain code
Supports White-box Testing
Essential for structural testing and path analysis
Evaluation
Low complexity → Easy to test and maintain
High complexity → More test cases, higher risk
Encourages developers to simplify code structure

2.

Clean Code & Documentation Standards
Introduction

Inconsistent coding practices and poor documentation reduce

readability, maintainability, and collaboration. A structured approach is required to ensure clean and well-documented code.

1. Clean Code Practices

a) Meaningful Naming

Use clear and descriptive names
Example: calculateInterest() instead of calc()

b) Small & Focused Functions

Each function should perform one task only

c) Code Modularity

Break system into reusable modules/classes

d) Consistent Coding Style

Follow indentation, spacing, and formatting standards

e) Avoid Code Duplication

Use reusable functions and libraries (DRY

principle)

f) Proper Error Handling

Use exceptions and validations to handle errors gracefully

g) Simplicity

Avoid overly complex logic; keep code easy to understand

2. Code Documentation Standards

a) Inline Comments

Explain complex logic, not obvious code

b) Function Documentation

Include:
Purpose
Parameters
Return values

c) API Documentation

Clearly define endpoints, inputs, and

outputs

d) Use Standard Formats

Javadoc, Doxygen, or Markdown

e) Maintain Updated Documentation

Keep docs synchroniped with code changes
3. Tools & Practices
Code reviews
Version control (Git)
Automated documentation tools
Conclusion

A structured approach combining clean coding practices and proper documentation ensures:

Better readability
Easier maintenance
Improved team collaboration

Q3. Top-down vs Bottom-up Integration

Testing
Introduction

Integration testing ensures that modules work together correctly. Two common strategies are top-down and bottom-up integration testing.

1. Top-down Integration Testing
Testing starts from top-level modules and moves downward

Process:

High-level modules tested first
Lower modules replaced by stubs

Example:
Testing main banking system first, while account/database modules are simulated

Advantages:

Early validation of system design
No need for

drivers

Disadvantages:

Requires many stubs
Low-level modules tested late
2. Bottom-up Integration Testing
Testing starts from low-level modules and moves upward

Process:

Lower modules tested first
Higher modules replaced by drivers

Example:
Testing database and transaction modules first, then integrating with UI

Advantages:

No need for stubs
Thorough testing of core

modules

Disadvantages:

Requires drivers
System-level logic tested late
3. Role of Test Drivers and Test Stubs
Test Stubs:
Used in top-down testing
Simulate lower-level modules
Test Drivers:
Used in bottom-up testing
Simulate higher-level modules

4.

Introduction

Before full release, software is tested in stages to ensure quality and user satisfaction. Alpha testing and Beta testing are important pre-release testing techniques.

1. Alpha Testing
Conducted internally by developers or testers
Performed in a controlled

environment

👉 Example:
Testing a banking application within the company to check functionality and detect bugs

Significance:

Identifies major defects early
Ensures core functionality works properly
Allows developers to fix issues before user exposure
2. Beta Testing
Conducted by real users in a real-world environment
Limited release to selected users

👉 Example:
Releasing a banking app to a small group of customers to test real usage scenarios

Significance:

Provides real user feedback
Identifies usability and performance issues
Helps improve user

experience

Comparison

Feature	Alpha Testing	Beta Testing
Performed By	Internal team	End users
Environment	Controlled	Real-world
Purpose	Find bugs	Gather feedback
Stage	Before beta	Before final release

Overall Importance

Alpha testing ensures technical correctness

Beta testing ensures user satisfaction and usability

Together, they reduce risks and improve product quality

Conclusion

Alpha and beta testing complement each other by ensuring the software is both technically sound and user-friendly, leading to successful product deployment.

Q5. Static Testing Techniques

Introduction

Static testing is performed without executing the code. It helps detect defects early, reducing cost and effort in later stages.

1. Walkthroughs

Informal review

process
Author explains code/ design to team

Example:
DevelOper explains login module logic to teammates

Benefits:

Easy to conduct
Encourages team understanding

2. Code Inspections
Formal and structured review
Conducted by experts using checklists

Example:
Reviewing code for syntax errors, logic flaws, and standards

Benefits:

Detects defects early
Improves code quality and consistency
3. Automated Static Analysis Tools
Tools analyze code

automatically

Example:
Finding unused variables, security vulnerabilities, or code smells

Benefits:

Fast and efficient
Detects issues humans may miss
Importance of Static Testing
Early Defect Detection
Errors found before execution
Cost Reduction
Fixing bugs early is cheaper
Improved Code Quality
Ensures adherence to standards
Better Maintainability
Clean and well-structured code
Reduced Testing Effort
Fewer defects during dynamic

testing

6.

Introduction

Accurate estimation of cost, effort, and development time is crucial for successful software project management. The Basic COCOMO (Constructive Cost Model), proposed by Barry Boehm, is widely used for early-stage estimation based on project size.

Basic COCOMO Model

The model estimates effort using the formula:

Effort (Person-Months) = $a \times (KLOC)^b$

Where:

KLOC = Thousands of Lines of Code
a, b = Constants depending on project type
Project Types &

Constants

Project Type	a	b
Organic (small, simple)	2.4	1.05
Semi-detached (medium)	3.0	1.12
Embedded (complex)	3.6	1.20

Development Time Estimation

Time (Months) = c × (Effort)
d

Where:

c, d are constants (approx. 2.5 and 0.38)

Example

For a project of 50 KLOC (Organic type):

Effort = 2.4 × (50)
1.05
≈130 person-months

Time = 2.5 × (130)
0.38
≈14 months

Impact on Resource Allocation

Team Size Planning

Helps decide number of developers